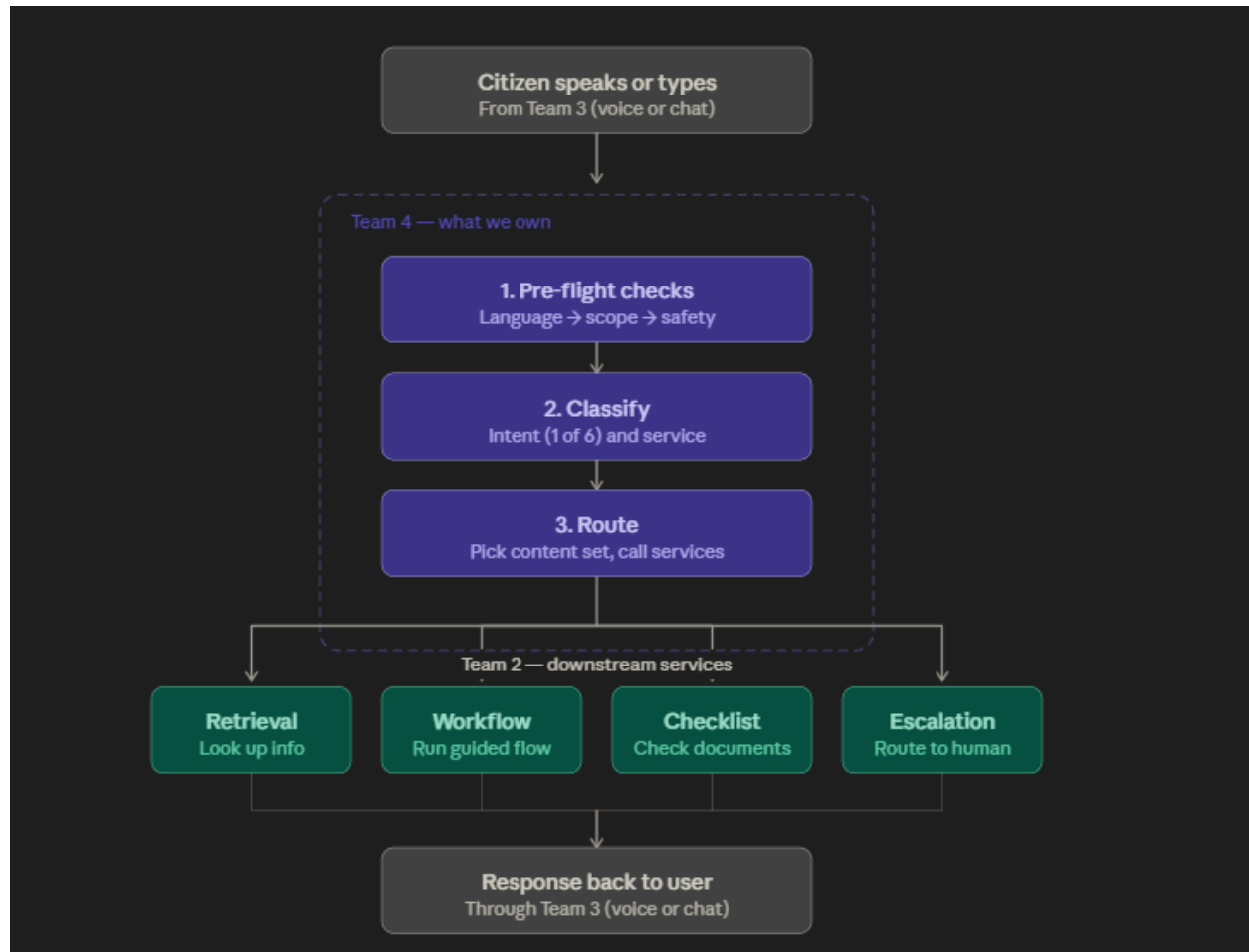


Team 4 — AI Agents & Workflow

Week 2 → Week 3 Handoff



Example 1 — English permit info question (the easy case)User says (typed or spoken via Team 3):

"What documents do I need for a timber permit?"

Return the following based on the request "What documents do I need for a timber permit?"

Team 4 outputs:

```
json{
```

```
"request_id": "req_8f2a91",
"session_id": "sess_4d7b22",
"timestamp": "2026-04-22T09:14:33Z",
"language": "en",
"in_scope": true,
"safety_flag": false,
"category": 1,
"category_label": "information_lookup",
"service": "Permit",
"entities": {
  "permit_type": "timber"
},
"content_set": "A",
"downstream_service": "knowledge_retrieval",
"confidence": 0.94
}
```

1. TL;DR

Team 1 has handed us validated user research. Team 2 has handed us the locked platform architecture. Team 4 has translated both into a working logic layer: a language → scope → safety → classification → service ID → content selection → workflow/retrieval → response pipeline, with explicit fallback and escalation paths.

Team 4 owns the **Intent Classification Service** in Team 2's platform diagram, plus the routing logic that sits between the Conversation Orchestrator and the downstream services (Knowledge Retrieval, Workflow Engine, Document Checklist, Status Lookup, Human Escalation, Response Generation).

Three prioritized scenarios (locked by Team 2, validated by Team 1):

1. Permit Application Assistance — Workflow Engine + Document Checklist driven
2. Health Information Access — Knowledge Retrieval driven, with guardrail policy enforced at Response Generation
3. Business Registration & Licensing — Workflow Engine (with eligibility branching) + Document Checklist driven

Decision pending team approval: lock the routing logic, the 6 request categories, and the 4 content sets below so Week 3 testing can begin Monday.

2. What we received from Team 1

From the international research track

- 6 semi-structured interviews surfaced friction across permits, health, business registration, public benefits, land/property, and transportation
- Phase 1 use-case shortlist selected Permit Application Assistance and Health Information Navigation; Business Registration was named as a Phase 2 candidate
- Personas span low-literacy rural citizens, semi-urban workers, small business applicants, elderly/assisted users, and government front-desk officers
- Journey maps confirm a consistent five-pattern friction across every scenario: no clear starting point, hard-to-read instructions, no step-by-step journey, no readiness validation, repeated office visits

From the Bhutan local survey (n = 20 valid out of 27)

- 88.9% of rural respondents said yes to a friendly voice assistant in Dzongkha
- 40.7% flagged poor or unstable internet — the workflow must tolerate dropped sessions and partial inputs, and SMS summaries are a baseline mitigation
- 22.2% cited English language / technical terms as a barrier — Dzongkha is the default, not optional
- 74.1% want clear information about requirements; 55.6% want step-by-step guidance — the dominant request type is informational, not transactional
- 44.4% want simple Q&A in Dzongkha
- 18.5% want to check the status of their application — this validates a dedicated Status / Service Lookup path

What this shapes in Team 4's design

- We are not building a transaction engine. We are building a classification and routing engine that sits inside Team 2's platform.
- Dzongkha is first-class. Every content set, fallback, and escalation message is authored in Dzongkha and English, with the language flag carried through the Session & Context Service.
- Connectivity-aware behavior is a baseline requirement. SMS summaries via the Notification Service close the loop when sessions drop.
- The Health scenario requires a guardrail policy enforced by the Response Generation Service before any Health response leaves the platform.

3. Team 4's Logic Layer — Locked Workflow

None

1. USER REQUEST enters via API Gateway / Channel API
2. ↓ (voice via Speech Processing Service, or chat directly)
3. ↓
4. Conversation Orchestrator Service receives the text
5. ↓

6. Orchestrator reads language + prior state from Session & Context Service
7. ↓
8. Orchestrator hands the request to Team 4's logic:
9. ↓
10. [1] LANGUAGE DETECTION → flag dz / en (write to Session & Context)
11. ↓
12. [2] SCOPE CHECK → in-scope or out-of-scope?
13. ↓
14. [3] GENERAL SAFETY CHECK → safe to answer or route to Human Escalation?
15. ↓
16. [4] INTENT CLASSIFICATION → which of 6 request categories?
17. (this is Team 2's Intent Classification Service – Team 4 owns it)
18. ↓
19. [5] SERVICE IDENTIFICATION → Permit / Health / Business / Other
20. ↓
21. [6] CONTENT SET SELECTION → which OpenSearch index slice + which
22. Aurora PostgreSQL rule set to load
23. ↓
24. [7] DOWNSTREAM SERVICE ROUTING → return decision to Orchestrator:
25. - Permit → Workflow Engine + Document Checklist
26. - Health → Knowledge Retrieval (Response Generation enforces
27. guardrail policy before output)
28. - Business → Workflow Engine (with eligibility branching) +
29. Document Checklist
30. - Status (any service) → Status / Service Lookup Service

```

31.         - Out-of-scope → Response Generation with redirect
           template
32.     ↓
33. [8] LOG DECISION → every classification + routing decision is
           written
34.         to the Audit & Compliance Service for traceability
35.     ↓
36. [9] RESPONSE GENERATION → grounded answer in user's language
37.     ↓
38. [10] OPTIONAL SMS SUMMARY → Notification Service sends recap
39.         (especially for Permit and Business workflows, and
           when
40.         connectivity is flagged as weak)
41.     ↓
42. [11] FALLBACK / ESCALATION → low confidence, unsupported, or
           sensitive
           → Human Escalation Service

```

Key principles:

- We classify before we retrieve, and we retrieve before we generate. This keeps responses grounded in approved content from OpenSearch and Aurora PostgreSQL, and prevents the model from inventing government procedures.
- Team 4's classifier is the brain of Team 2's Intent Classification Service. Routing decisions flow back to the Conversation Orchestrator, which calls the actual downstream services.
- The Health guardrail is a policy applied inside Response Generation when service == Health. It is non-optional but it is not a separate service.
- Eligibility logic for Business Registration lives inside the Workflow Engine Service as decision-tree branching. It is not a separate service.
- Every decision is logged to the Audit & Compliance Service.

4. In-Scope vs. Out-of-Scope (Phase 1)

In-scope for the prototype

Permit Application Assistance

- Information requests about permit procedures (Knowledge Retrieval)

- Document checklists per permit type (Document Checklist & Validation)
- Step-by-step guidance (Workflow Engine)
- Readiness validation before office visit (Document Checklist & Validation)
- Status-check guidance (Status / Service Lookup)
- SMS summary of checklist (Notification Service)

Health Information Access

- Public health information navigation in plain Dzongkha and English (Knowledge Retrieval)
- Where to go, what service applies, what the next step is (Knowledge Retrieval)
- Simplification of formal or technical health language (Response Generation)
- Facility / service-location guidance (Knowledge Retrieval)
- Guardrail policy enforced at Response Generation

Business Registration & Licensing

- End-to-end procedural explanation (Knowledge Retrieval)
- Business-type identification questions (Workflow Engine)
- Eligibility-path determination (Workflow Engine — decision-tree branching)
- Document checklist per business type (Document Checklist & Validation)
- Status-check guidance (Status / Service Lookup)
- SMS summary of next steps (Notification Service)

Cross-cutting

- Dzongkha or English voice and chat input
- Connectivity-aware short responses
- Language flag and workflow progress carried in Session & Context

Out-of-scope this sprint

- Submitting applications on behalf of the user
- Live integration with G2C, MoH, or MoICE backend Government Service Systems
- Land tax, insurance, transportation, public benefits, security clearance
- Medical diagnosis, treatment recommendations, or any clinical advice (guardrail policy blocks this)
- Legal interpretation of eligibility decisions
- Payment processing
- Persistent personal data storage between sessions (Session & Context only holds short-lived state)

Out-of-scope requests get a polite redirect with the right next step, not a flat refusal. See Section 8.

5. The 6 Request Categories

Every user request is classified into exactly one of these:

Category	Example utterance	Routes to
Information lookup	"What documents do I need for a sand permit?"	Knowledge Retrieval Service
Step-by-step guidance	"How do I register my small shop?"	Workflow Engine Service
Eligibility / readiness	"Am I ready to go to the office?"	Document Checklist & Validation, or Workflow Engine for Business
Status / where-to-check	"Where do I check my permit status?"	Status / Service Lookup Service
Out-of-scope service	"How do I pay my land tax?"	Response Generation with redirect template
Unclear / ambiguous	"I need help with government"	Clarifying question loop

Confidence threshold for classification: 0.70. Below that → category 6 (clarifying loop, max 2 retries before fallback to Human Escalation).

6. Content Sets (aligned with Team 2's data stores)

Each content set is a logical bundle of approved material indexed into **OpenSearch** for retrieval, with associated rules stored in **Aurora PostgreSQL**. Session state for any active workflow is held in **DynamoDB**. Source files (PDFs, audio guides, KB drops) live in **S3**. Team 4 does not

own any of these stores — Team 2 does — but our routing logic decides which slice gets loaded for any given request.

Content Set A — Permit Application Assistance

- **OpenSearch slice:** permit FAQs, official descriptions, common errors and remedies, approving authority, typical timelines, status-check pathway
- **Aurora PostgreSQL slice:** permit eligibility rules, document requirements per permit type, workflow definitions for the Workflow Engine
- **Maps to:** Team 2 Scenario 1 container diagram

Content Set B — Health Information Access

- **OpenSearch slice:** official public health content, simplified explanations, facility navigation rules, service-type mapping, next-step rules
- **Aurora PostgreSQL slice:** guardrail policy (what cannot be answered, escalation triggers), Health Worker escalation message templates
- **Maps to:** Team 2 Scenario 2 container diagram
- Guardrail policy is loaded by Response Generation Service before any Health response is returned

Content Set C — Business Registration & Licensing

- **OpenSearch slice:** procedural FAQs, business-type definitions, office sequence, document requirements per business type, status-check pathway
- **Aurora PostgreSQL slice:** licensing rules, eligibility decision tree, workflow definitions with branching for the Workflow Engine
- **Maps to:** Team 2 Scenario 3 container diagram

Content Set D — System / Meta / General Navigation

- **OpenSearch slice:** "what can you do?" responses, language switch prompts, light agency directory for out-of-scope redirects
- **Aurora PostgreSQL slice:** escalation message templates, helpline numbers, fallback message library

Each content set is authored in Dzongkha and English. Team 1 validates factual accuracy before Week 3 testing. Authors hand drafts to Team 2 for ingestion into OpenSearch and Aurora PostgreSQL.

7. Routing Logic — Decision Rules

None

43. // Read from Session & Context Service first

44. IF language == unknown pick language	→ ask user to repeat /
45. IF general_safety_flag == true Escalation Service	→ route to Human
46. IF category == 5 → redirect template (Set D)	→ Response Generation
47. IF category == 6 (max 2 retries)	→ clarifying question
48. IF retries_exceeded Service	→ Human Escalation
49.	
50. // Permit	
51. IF category == 1 AND service == Permit (Set A)	→ Knowledge Retrieval
52. IF category == 2 AND service == Permit A)	→ Workflow Engine (Set
53. IF category == 3 AND service == Permit Validation (Set A)	→ Document Checklist &
54. IF category == 4 AND service == Permit Lookup	→ Status / Service
55. (SMS summary, optional)	→ Notification Service
56.	
57. // Health	
58. IF service == Health (Set B)	→ Knowledge Retrieval
59. enforces guardrail policy	→ Response Generation
60. IF guardrail blocks Service (Health Worker)	→ Human Escalation
61. IF category == 1 response	→ standard retrieval
62. IF category == 2 OR 4 facility/next-step navigation	→ retrieval +
63.	
64. // Business	

```

65. IF category == 1 AND service == Business → Knowledge Retrieval
    (Set C)
66. IF category == 2 AND service == Business → Workflow Engine
    (Set C, with eligibility branching)
67.                                     → Document Checklist
    & Validation
68. IF category == 3 AND service == Business → Workflow Engine
    (eligibility branch, Set C)
69. IF category == 4 AND service == Business → Status / Service
    Lookup
70.                                     → Notification
    Service (SMS summary, optional)
71.
72. // Always

EVERY decision                                     → Audit & Compliance
Service (log)

```

8. Fallback & Apology Messages (drafts — Team 2 to polish tone)

Fallback — low confidence "I'm not fully sure I understood. Could you tell me again, slowly? For example, you can say I want to apply for a timber permit, I want to know where to get a vaccine, or I want to register a small business."

Out-of-scope — service we don't cover yet "I can help with permit applications, health information, and business registration right now. For [land tax / insurance / etc.], please visit your nearest Community Centre or call the G2C helpline."

Health guardrail — clinical advice "I cannot give medical advice. For anything about your symptoms, treatment, or medicine, please speak to a health worker or visit your nearest Basic Health Unit. I can help you find where to go."

Connectivity-aware fallback "Your connection seems weak. I'll keep my answers short. Please ask one question at a time. I can also send a summary by SMS."

Escalation — sensitive or stuck "This is something a person should help you with. Please visit your nearest Community Centre, or I can share the helpline number."

Clarifying question loop "Could you tell me which service you need help with — a permit, health information, or registering a business?"

9. Open Tasks — Volunteers Needed

We need four people to take ownership of the work below. Pick one. Add your name on Slack in the Team 4 channel by end of day. Each role is roughly 4–6 hours over Week 3, plus the team syncs.

Task 1 — Content Set Authoring Lead

What this means in plain language: Our system can only answer questions if we feed it the right reference material. A "content set" is a logical bundle of plain-text documents and rules that Team 2 will ingest into OpenSearch (for search) and Aurora PostgreSQL (for rules). We need four of these sets written.

What you'll actually do:

- Write **Set A (Permit)**: documents listing what timber/sand/stone permits exist, what documents each requires, what the steps are, who approves it, how long it takes, and the rules that the Workflow Engine and Document Checklist Service will use
- Write **Set B (Health)**: documents explaining where citizens go for common health needs, how to navigate facilities, plus the guardrail policy listing what the AI is not allowed to answer (e.g. "do not diagnose, do not recommend medicine"). The guardrail will be enforced by Response Generation Service
- Write **Set C (Business Registration)**: business types, eligibility decision tree (which Workflow Engine will execute as branching), document checklists, and office sequence
- Write **Set D (System/Meta)**: short responses for "what can you do?", language switch prompts, polite redirects for out-of-scope services, helpline numbers
- Pull source material from official Bhutan government websites
- Send drafts to Team 1 (international) to fact-check the procedures
- Hand finished English drafts to the Dzongkha translation lead, then to Team 2 for ingestion

You don't need to be: A subject-matter expert. You're an editor and organizer. Team 1 validates the facts, Team 2 ingests the content.

Output: Four Markdown or Google Doc files, one per set, in English, structured into "knowledge content" (for OpenSearch) and "rules" (for Aurora PostgreSQL).

Task 2 — Dzongkha Translation Lead

What this means in plain language: Everything the user sees or hears must work in Dzongkha — that's the whole point of the project (88.9% of the Bhutan survey said yes to a Dzongkha voice assistant). Once Task 1 produces English content sets, those need to become Dzongkha versions. Same for the fallback messages and the guardrail responses.

What you'll actually do:

- Take the four English content sets from Task 1 and produce Dzongkha versions
- Translate the fallback messages (low confidence, out-of-scope, connectivity-aware, escalation, clarifying questions) — Section 8
- Translate the Health guardrail message
- Coordinate with native Dzongkha speakers on the team — most likely the Bhutan local research group
- Flag terms that don't translate cleanly and decide whether to keep them in English or coin a Dzongkha equivalent
- Make sure the tone is friendly and simple — not formal government-speak, since target users are rural farmers with low formal education

You don't need to be: A professional translator. You need to be able to coordinate native speakers and review their work for consistency.

Output: Dzongkha versions of all four content sets and all fallback messages, side-by-side with the English originals.

Task 3 — Classifier Prototype + Test Set Builder

What this means in plain language: The whole logic layer in Section 3 is just an idea on paper until someone codes it up and tests whether it actually works. This task is the engineering heart of Week 3. We need a working Python notebook that takes a user question as input and outputs (a) which of the 6 categories it is, (b) which service it's about, and (c) which downstream service the Orchestrator should call. Then we measure how often it gets that right.

What you'll actually do:

- Build a Jupyter/Colab notebook that implements the Intent Classification Service — you can use a simple LLM API call with a prompt that includes the 6 categories, 3 services, and routing rules from Section 7
- Write 30–50 sample user questions per scenario (~100–150 total) in both Dzongkha and English. Include tricky cases: ambiguous questions, out-of-scope questions, mixed-language questions
- For each test question, write down what the correct category, service, and downstream-service routing should be (this is your ground truth)
- Run all the test questions through the classifier and measure: what % did it get right? Where did it fail?
- Document the failure patterns
- Expose the classifier as a simple Python function so Task 4 can plug it into the Orchestrator contract

You don't need to be: An ML researcher. This is prompt engineering plus basic accuracy measurement, not model training.

Output: A notebook (`classifier_prototype.ipynb`), a test set file (`test_questions.csv`), and a one-page failure analysis.

Task 4 — Integration + Demo Lead

What this means in plain language: Team 4 doesn't live in isolation. Team 2 has built the platform (API Gateway, Conversation Orchestrator, downstream services, data stores), and Team 3 has built the voice/chat front end (Channels, Speech Processing, Voice Channel Adapter). Our classifier from Task 3 has to plug into both via the Orchestrator. This person makes sure the wiring works and that we have something real to show on Demo Day (Day 15).

What you'll actually do:

- Talk to Team 2 and write down exactly what format the Conversation Orchestrator expects from the Intent Classification Service (e.g. `{language, category, service, content_set_id, downstream_service}`) and what it sends back. This is the contract
- Talk to Team 3 and confirm the format of speech-to-text output coming out of the Speech Processing Service
- Take the classifier function from Task 3 and connect it to the Orchestrator contract on the input side and the downstream services on the output side
- Confirm that every classification decision is being logged to the Audit & Compliance Service
- When things break (and they will), keep a running log of failures. This log feeds the Feasibility Report in Week 4
- Prepare a 5-minute demo for Day 15: someone speaks a Dzongkha or English question into Team 3's interface, our classifier picks it up via the Orchestrator, routes to the right downstream service and content set, response comes back through Speech Processing, user hears it

You don't need to be: A senior engineer. You need to be organized, communicative, and willing to chase down two other teams.

Output: Two short interface contract documents (one with Team 2, one with Team 3), a working integration script, a failure log, and the Day 15 demo.

10. Risks & Limitations (early flag for the Feasibility Report)

1. No live integration with Government Service Systems (G2C, MoH, MoICE). We can guide users to the right place but cannot verify status, submit, or pull live data through the Status / Service Lookup Service in any real sense. Phase 2 dependency.
2. Dzongkha speech recognition quality at rural-dialect level is unproven. Team 3 is testing — if Speech Processing accuracy is poor, the workflow still functions on chat, but the voice promise weakens.
3. Content sets are currently built from public web sources and need official validation before Phase 2 ingestion into OpenSearch.
4. Connectivity. 40.7% of target users have unstable internet. The workflow tolerates this, and the Notification Service SMS summary is a baseline mitigation, but real testing is needed.
5. Health domain risk. Even with the guardrail policy enforced at Response Generation, a misinterpreted question could surface unsafe content. The guardrail will be evaluated with worst-case adversarial prompts during Week 3.
6. Business Registration complexity is high. Phase 1 prototype is scoped to a single common business type (e.g., small retail) with the depth limit documented.
7. Audit logging volume. Every classification decision is written to the Audit & Compliance Service. For the prototype this is fine; at scale, log volume needs review.

11. Week 3 Plan

- Mon–Tue: Finalize content Sets A, B, C, D with Team 1 validation; Dzongkha translation pass; hand to Team 2 for ingestion into OpenSearch + Aurora PostgreSQL
- Tue–Wed: Build the classifier prototype (Python notebook is sufficient, no UI)
- Wed–Thu: Run 30–50 sample user questions per scenario through the classifier; measure accuracy per category and per service
- Wed–Thu: Adversarial pass on the Health guardrail policy
- Thu–Fri: Sync with Team 3 — pipe Speech Processing output into the classifier via the Orchestrator contract; capture failures
- Fri: Document failures and edge cases for the Feasibility Report

12. Decisions needed today

1. Approve the 6-category classification taxonomy in Section 5
 2. Approve the 4 content sets in Section 6
 3. Confirm out-of-scope list in Section 4
 4. Approve the Health guardrail policy as a non-optional check inside Response Generation
 5. Confirm every classification decision is logged to the Audit & Compliance Service
 6. Volunteer sign-ups against Section 9 tasks
- 73.